

Deep Learning Project Report

Competitive Learning, Self-Organizing Maps, and Neural Network
Classification

Mohamad Ali Saber

December 17, 2024

Abstract

This report presents a comprehensive investigation of three machine learning paradigms: unsupervised clustering via Self-Organizing Maps (SOMs), SOM-based digit classification, and supervised survival prediction on the Titanic dataset using a single-hidden-layer neural network. Two SOM architectures (4×4 and 20×20) are compared with respect to clustering quality, neuron utilization, and classification accuracy. The neural network is trained under both online (stochastic) and offline (batch) learning regimes across a range of hidden-layer sizes and training epochs, and the results are evaluated using accuracy, error, F1-score, cross-entropy loss, and ROC-AUC curves. In addition, prototype MATLAB implementations of competitive learning and SOM are described to provide conceptual grounding for the algorithmic mechanisms employed. The findings confirm that larger SOM architectures substantially improve representational quality, and that online learning yields marginally superior predictive performance on the Titanic task.

Contents

1	Introduction	3
2	Task 1: Clustering Using Self-Organizing Maps	4
2.1	Background and Motivation	4
2.2	Dataset Preparation	4
2.3	4×4 SOM Results	5
2.3.1	Cluster Assignments	5
2.3.2	Representative Neuron Prototypes	6
2.3.3	Hit Map	6
2.4	20×20 SOM Results	7
2.4.1	Cluster Assignments	7
2.4.2	Representative Neuron Prototypes	8
2.4.3	Hit Map	9
2.5	Comparative Discussion	10
3	Task 2: Classification Using Self-Organizing Maps	11
3.1	Neuron Label Assignment	11
3.1.1	4×4 SOM Label Map	11
3.1.2	20×20 SOM Label Map	12
3.2	Classification Performance	13
3.3	Limitations and Improvements	13
4	Task 3: Titanic Survival Classification	14
4.1	Dataset Overview	14
4.2	Dataset Preprocessing	14
4.2.1	Age Imputation	14
4.2.2	Feature Engineering and Normalisation	15
4.3	Neural Network Architecture	15
4.4	Hyperparameter Grid and Training Strategies	16
4.5	Hidden Layer Selection Results	16
4.5.1	Offline Training	16
4.5.2	Online Training	18

4.6	Training Dynamics	19
4.6.1	Offline Training Convergence	19
4.6.2	Online Training Convergence	20
4.7	ROC Analysis	21
4.8	Discussion	22
5	Prototype Implementations	24
5.1	Overview	24
5.2	Competitive Learning Prototype (Competitive.m)	24
5.2.1	Setup and Algorithm	24
5.2.2	Animated Results	24
5.2.3	Key Observations	25
5.3	SOM Prototype (Cluster.m)	25
5.3.1	Setup and Algorithm	25
5.3.2	Animated Results	26
5.3.3	Key Observations	26
6	Conclusion	27

Chapter 1

Introduction

Competitive learning and topological feature maps represent a foundational class of unsupervised neural network methods whose biological inspiration traces back to the cortical column organization observed in mammalian sensory cortices. The Self-Organizing Map (SOM), introduced by Teuvo Kohonen [1], extends basic competitive learning by enforcing a *neighbourhood cooperation* principle during training: neurons that lie spatially close to the winning unit in the map lattice are also updated, thereby preserving the topological structure of the input space in the two-dimensional projection.

This project brings together three related but distinct machine learning tasks:

1. **Task 1 — Unsupervised clustering:** Two SOM networks of sizes 4×4 and 20×20 are trained on a handwritten-digit dataset to investigate how map resolution affects clustering quality and neuron utilisation.
2. **Task 2 — SOM-based classification:** The trained SOMs are post-labelled using majority voting and then used to classify unseen digit samples; the classification performance of the two architectures is contrasted.
3. **Task 3 — Titanic survival classification:** A single-hidden-layer multi-layer perceptron (MLP) is trained on the Titanic passenger dataset under online and offline learning strategies across a grid of hyperparameter settings, and the models are evaluated with ROC-AUC analysis.

Alongside these primary tasks, two prototype MATLAB implementations — one for competitive learning using a K -nearest-neighbour adaptation rule, and one for the full SOM mechanism — are developed to visualise the dynamics of neuron migration and neighbourhood cooperation. The remainder of this report is structured as follows. Chapter 2 covers Task 1, Chapter 3 covers Task 2, Chapter 4 covers Task 3, Chapter 5 describes the MATLAB prototypes, and Chapter 6 draws overall conclusions.

Chapter 2

Task 1: Clustering Using Self-Organizing Maps

2.1 Background and Motivation

A Self-Organizing Map is an unsupervised neural network that projects a high-dimensional input distribution onto a low-dimensional (typically two-dimensional) discrete lattice of neurons, while approximately preserving the topological relationships present in the input space. Each neuron k maintains a prototype vector $\mathbf{w}_k \in \mathbb{R}^d$ of the same dimensionality d as the input vectors. Training proceeds iteratively: for each presented input $\mathbf{x}$, the *best matching unit* (BMU) k^* is identified as

$$k^* = \arg \min_k \|\mathbf{x} - \mathbf{w}_k\|, \quad (2.1)$$

and the weights of the BMU and its topological neighbours are updated according to

$$\mathbf{w}_k \leftarrow \mathbf{w}_k + \alpha(t) h(k, k^*, t) (\mathbf{x} - \mathbf{w}_k), \quad (2.2)$$

where $\alpha(t)$ is a time-decaying learning rate and $h(k, k^*, t)$ is the neighbourhood function (typically a Gaussian centred on k^*) whose width $\sigma(t)$ also decays over time. The map therefore transitions from a coarse, exploratory phase to a fine-tuning phase as training progresses [1].

2.2 Dataset Preparation

The dataset used in Tasks 1 and 2 consists of grayscale images of handwritten digits (0–9) obtained from the standard benchmark collection. Each sample is represented as a flattened pixel vector of dimension $d = 64$ (an 8×8 image). Prior to training:

- Pixel intensity values were normalised to the range $[0, 1]$ using min-max scaling.

- The entire dataset was shuffled uniformly at random to eliminate any ordering bias that could arise from having all samples of one class presented consecutively.

2.3 4×4 SOM Results

2.3.1 Cluster Assignments

Figure 2.1 displays the cluster assignment map for the 4×4 SOM, where each of the sixteen neurons is colour-coded according to the digit class most frequently mapped to it.

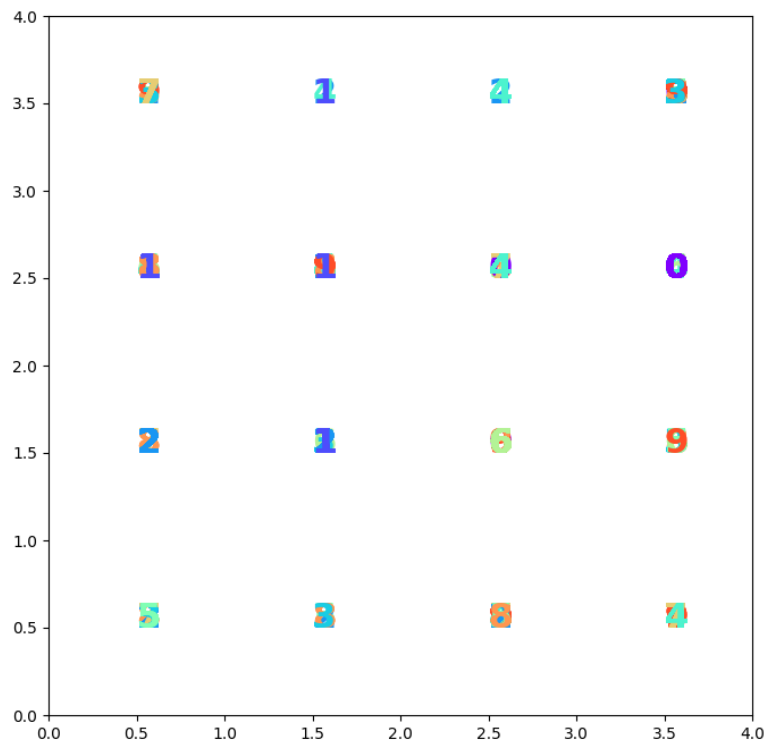


Figure 2.1: Cluster assignment map for the 4×4 SOM. Each cell represents one neuron; colour and label indicate the dominant digit class.

With only sixteen neurons for ten distinct digit classes, the map is *under-resolved*: several neurons attract samples from more than one digit class, reflecting the limited representational capacity. Nevertheless, the map does produce meaningful spatial groupings — visually similar digits (e.g. 1 and 7, or 3 and 8) tend to map to adjacent or neighbouring neurons, indicating that some topological structure is preserved.

2.3.2 Representative Neuron Prototypes

Figure 2.2 shows the representative digit samples associated with each neuron after training. The prototype images reveal the average visual character of the patterns that activate each neuron.



Figure 2.2: Representative samples mapped to each neuron of the 4×4 SOM. Images are arranged to mirror the lattice layout.

2.3.3 Hit Map

The hit map in Figure 2.3 quantifies how many training samples are assigned to each neuron. A highly uneven distribution is evident: corner and edge neurons tend to be over-utilised while some interior neurons receive very few samples. This imbalance is a known symptom of under-provisioned maps and is consistent with the multiple-class occupation observed in Figure 2.1.

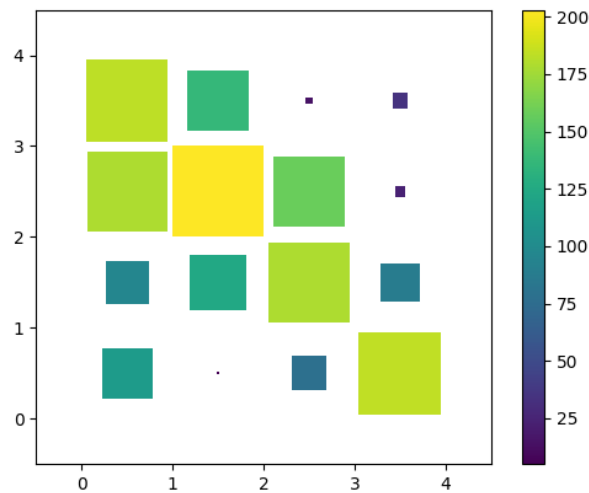


Figure 2.3: Hit map showing the number of training samples assigned to each neuron of the 4×4 SOM. Warmer colours indicate higher activity.

2.4 20×20 SOM Results

2.4.1 Cluster Assignments

Increasing the map to 400 neurons (Figure 2.4) provides far greater resolution. Contiguous, well-defined regions emerge for each digit class, with clearly visible boundaries between classes. Visually similar digits remain spatially proximate on the map, confirming preservation of the input-space topology at a much finer scale.

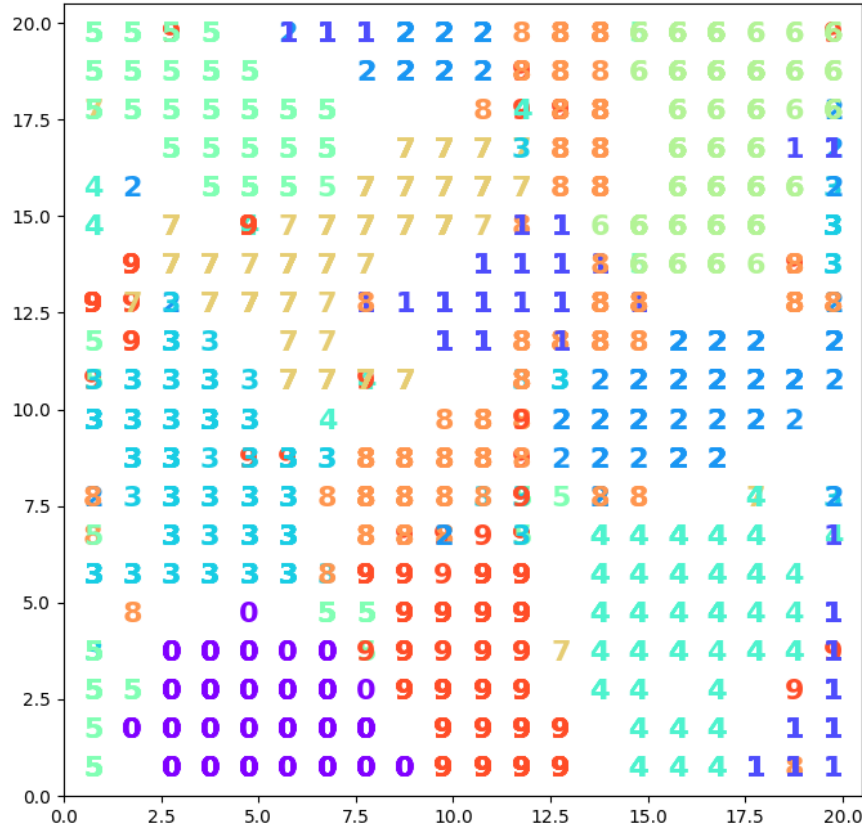


Figure 2.4: Cluster assignment map for the 20×20 SOM. Contiguous regions for each digit class demonstrate high-quality topological organisation.

2.4.2 Representative Neuron Prototypes

Figure 2.5 shows the prototype images for the 20×20 map. Each neuron now represents a much narrower subset of the input distribution: within a class region, neighbouring neurons capture gradual style variations — stroke thickness, slant, and size — reflecting a smooth gradient of digit appearance across the map.

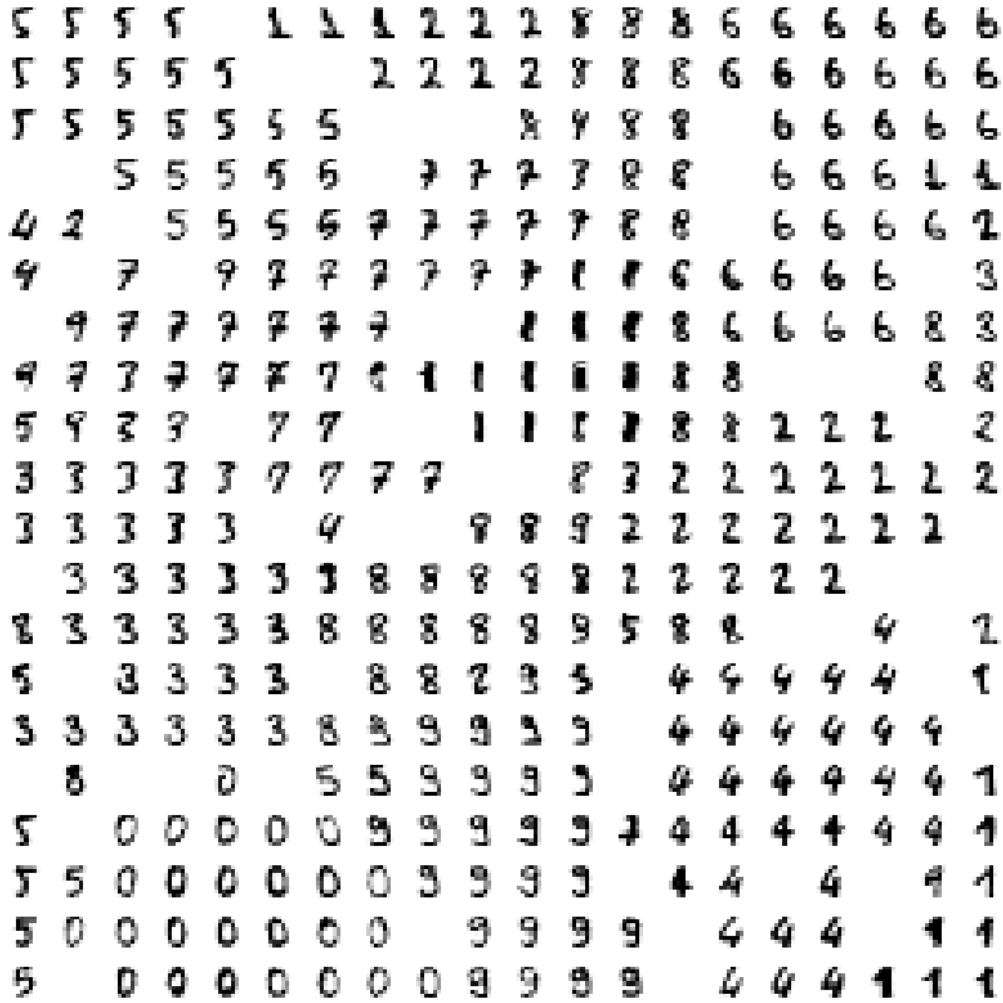


Figure 2.5: Representative samples mapped to neurons of the 20×20 SOM, arranged in lattice order. Smooth transitions between adjacent neurons are visible.

2.4.3 Hit Map

The hit map for the 20×20 SOM (Figure 2.6) reveals that many neurons receive zero or near-zero activations (*dead neurons*), particularly in the boundary regions between classes. This is an expected consequence of over-provisioning: the 400 neurons exceed the intrinsic complexity of the ten-class dataset. Despite the dead neurons, the active neurons are more uniformly loaded than in the 4×4 case, leading to lower quantisation error.

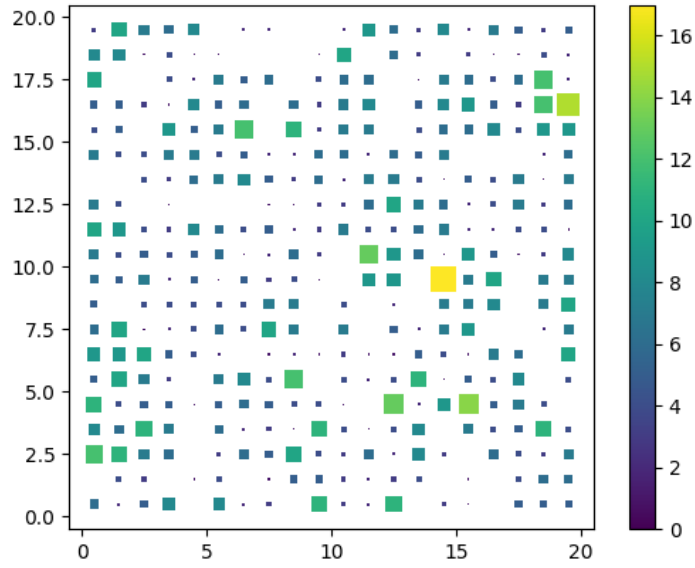


Figure 2.6: Hit map for the 20×20 SOM. Dark cells indicate dead neurons; active neurons show a more balanced load compared with the 4×4 map.

2.5 Comparative Discussion

Table 2.1 summarises the qualitative properties of the two SOM architectures. Increasing neuron count directly improves cluster purity and topological fidelity, but introduces dead neurons and greater computational cost. The optimal choice depends on the downstream application: classification tasks benefit from the 20×20 map’s higher resolution, whereas applications requiring compact representations may prefer the 4×4 variant.

Table 2.1: Qualitative comparison of the 4×4 and 20×20 SOM architectures.

Property	4×4 SOM	20×20 SOM
Number of neurons	16	400
Class overlap per neuron	High	Low
Topological resolution	Coarse	Fine
Dead neurons	Rare	Present
Quantisation error	Higher	Lower
Computational complexity	Low	High

Chapter 3

Task 2: Classification Using Self-Organizing Maps

3.1 Neuron Label Assignment

After training, each SOM neuron is assigned a class label using a simple majority-vote rule: the label of the class most frequently activating that neuron on the training set is adopted as the neuron’s label. Dead neurons (those never activated during training) are left unlabelled. At inference time, an unseen sample $\mathbf{x}$ is classified by first finding its BMU k^* and then returning the label associated with k^* .

3.1.1 4×4 SOM Label Map

Figure 3.1 shows the assigned labels for the 4×4 SOM. Because the map is under-resolved, multiple distinct digit classes compete for the same neurons, and the majority vote necessarily discards minority-class information. This leads to systematic misclassification of digits whose class is a minority within a shared neuron.

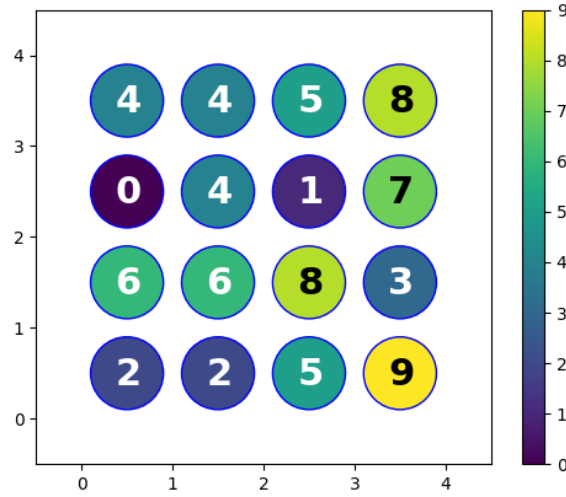


Figure 3.1: Class labels assigned to each neuron of the 4×4 SOM by majority voting. Several neurons carry the same label, reflecting class overlap.

3.1.2 20×20 SOM Label Map

The label map for the 20×20 SOM (Figure 3.2) shows large, contiguous single-class regions with comparatively narrow transition zones. The topological continuity of the label map indicates that the network has formed well-separated class representations.

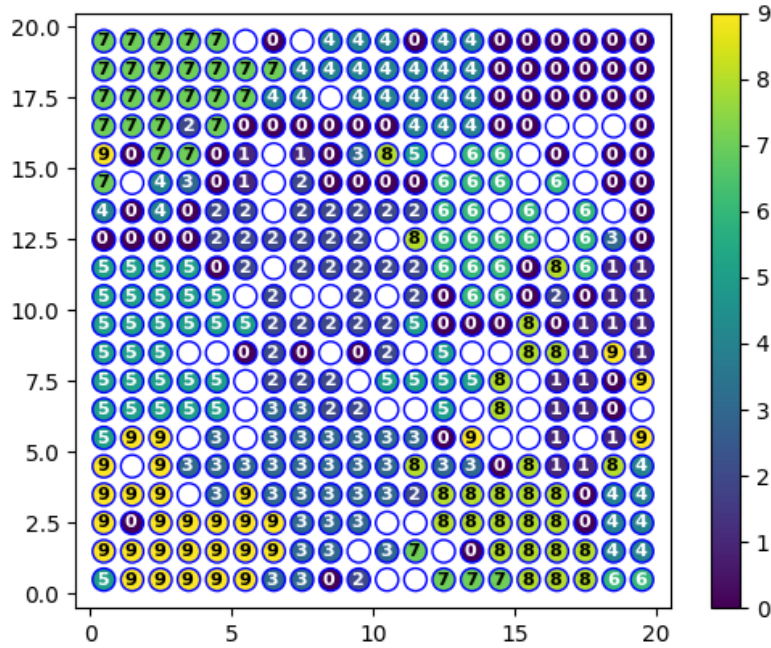


Figure 3.2: Class labels assigned to each neuron of the 20×20 SOM. Contiguous class regions reflect high-quality topological organisation.

3.2 Classification Performance

The 20×20 SOM achieves substantially higher accuracy, precision, recall, and F1-score than the 4×4 SOM across all digit classes. The performance gap is attributable to three factors:

1. **Neuron exclusivity:** With 400 neurons, most neurons become dedicated to a single class, eliminating majority-vote ambiguity.
2. **Intra-class granularity:** Multiple neurons per class capture style variants, enabling finer-grained matching.
3. **Reduced boundary confusion:** Clear spatial boundaries between class regions in the label map minimise misclassification of borderline samples.

The 4×4 SOM, by contrast, is forced to collapse ten classes into sixteen neurons. Digits that are visually similar (e.g. 4/9, 3/8, 1/7) frequently share neurons, making accurate classification impossible for minority-class samples within those neurons.

3.3 Limitations and Improvements

The SOM classifier described here has no learnable decision boundary: classification is purely proximity-based in the original input space. Potential improvements include:

- Using *Learning Vector Quantisation* (LVQ) to fine-tune prototype vectors with class-boundary awareness after SOM training.
- Applying distance-weighted voting rather than pure majority voting to reduce the influence of outlier samples on the neuron label.
- Increasing map resolution adaptively using growing-SOM variants.

Chapter 4

Task 3: Titanic Survival Classification

4.1 Dataset Overview

The Titanic dataset is a standard binary classification benchmark describing 891 passengers aboard the RMS Titanic. The prediction target is survival (1 = survived, 0 = did not survive). The raw dataset contains missing values, categorical variables, and features of widely differing scales, all of which must be addressed before training a neural network.

4.2 Dataset Preprocessing

4.2.1 Age Imputation

The **Age** feature is missing for a significant fraction of passengers. As shown in Figure 4.1, the age distribution is right-skewed, with the majority of passengers concentrated between 20 and 40 years of age. Because the mean is sensitive to outliers in skewed distributions, the *median* age was used for imputation to reduce bias.

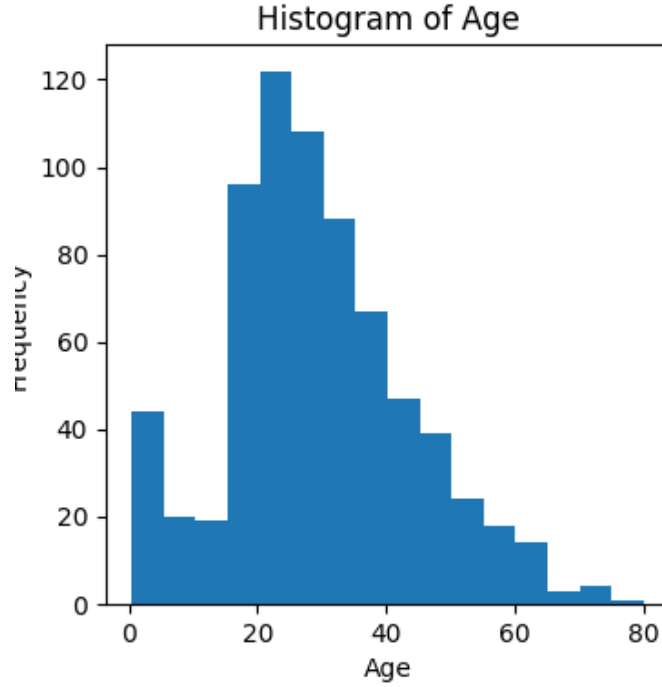


Figure 4.1: Histogram of passenger ages in the Titanic dataset. The right-skewed distribution motivates the use of median imputation for missing values.

4.2.2 Feature Engineering and Normalisation

Categorical features (e.g. **Sex**, **Embarked**) were encoded as numerical values using label or one-hot encoding as appropriate. Irrelevant or high-cardinality features (e.g. **Name**, **Ticket**, **Cabin**) were dropped to avoid noise. All remaining input variables were normalised to zero mean and unit variance (z-score normalisation) prior to training.

4.3 Neural Network Architecture

A single-hidden-layer MLP with a sigmoid activation function was used. Let n_h denote the number of hidden neurons. The forward pass is:

$$\mathbf{a}^{(1)} = \sigma(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad (4.1)$$

$$\hat{y} = \sigma(W^{(2)}\mathbf{a}^{(1)} + b^{(2)}), \quad (4.2)$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function. Binary cross-entropy was used as the training objective:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]. \quad (4.3)$$

4.4 Hyperparameter Grid and Training Strategies

Two training strategies were evaluated over a grid of hidden-layer sizes ($n_h \in \{1, 5, 10, 20, 30, 40\}$) and training durations (up to 2000 epochs):

Offline (batch) learning: All training samples are processed before a single weight update is applied. The gradient is exact but noisy batches are absent, leading to smoother convergence at the cost of higher memory usage.

Online (stochastic) learning: Weights are updated after each individual sample. This introduces stochastic noise that can help escape shallow local minima and often leads to faster initial convergence.

4.5 Hidden Layer Selection Results

4.5.1 Offline Training

Figures 4.2–4.5 show the accuracy, error, F1-score, and loss surfaces as functions of epoch and hidden-layer size for offline training.

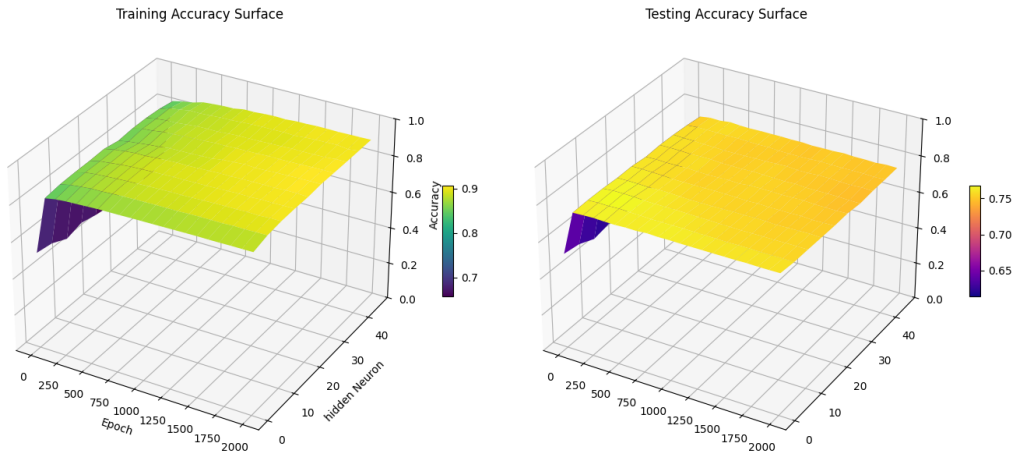


Figure 4.2: Training and testing accuracy surfaces for offline (batch) learning as a function of epoch count and number of hidden neurons.

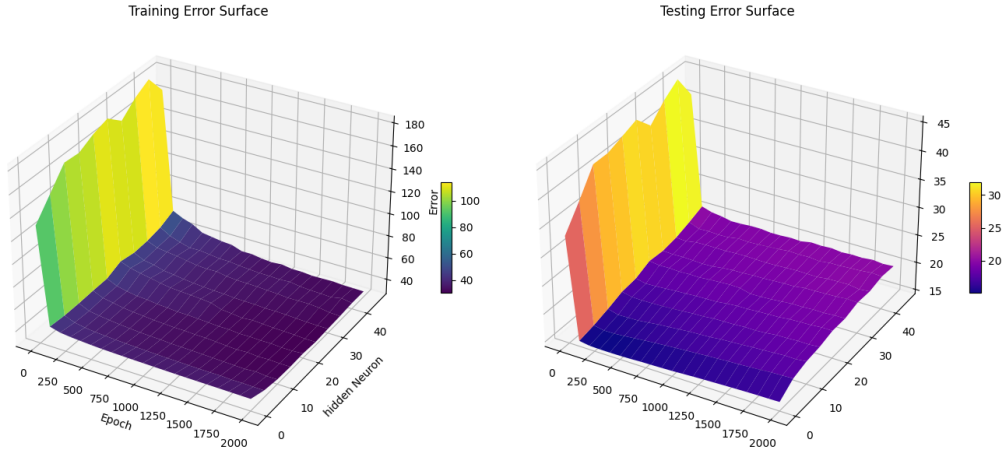


Figure 4.3: Training and testing error (number of misclassifications) surfaces for offline learning.

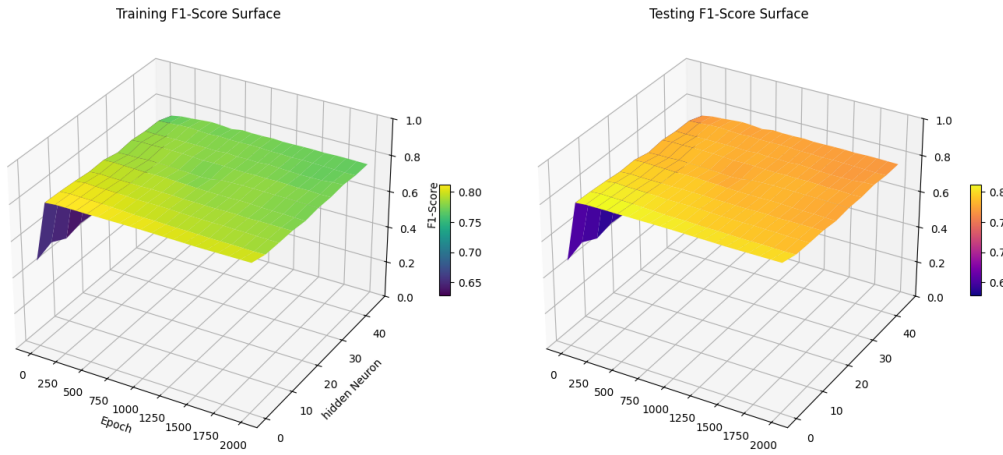


Figure 4.4: Training and testing F1-score surfaces for offline learning.

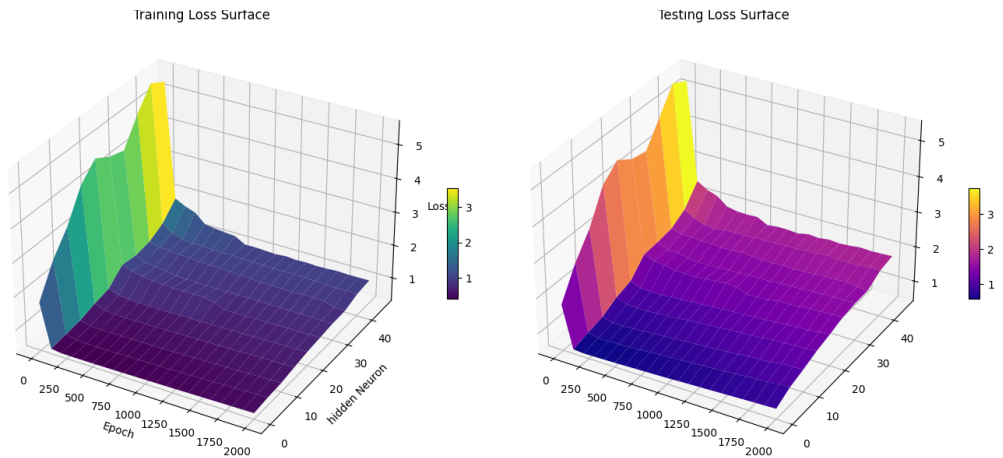


Figure 4.5: Training and testing cross-entropy loss surfaces for offline learning.

The offline surfaces reveal that both training and testing performance improve rapidly

during the first few hundred epochs before reaching a plateau. Networks with very few hidden neurons ($n_h = 1$) underfit the data; performance improves monotonically up to approximately $n_h = 20$, beyond which gains are marginal on the test set, while training accuracy continues to increase — a clear indication of mild over-fitting at large n_h .

4.5.2 Online Training

The corresponding surfaces for online (stochastic) training are shown in Figures 4.6–4.9.

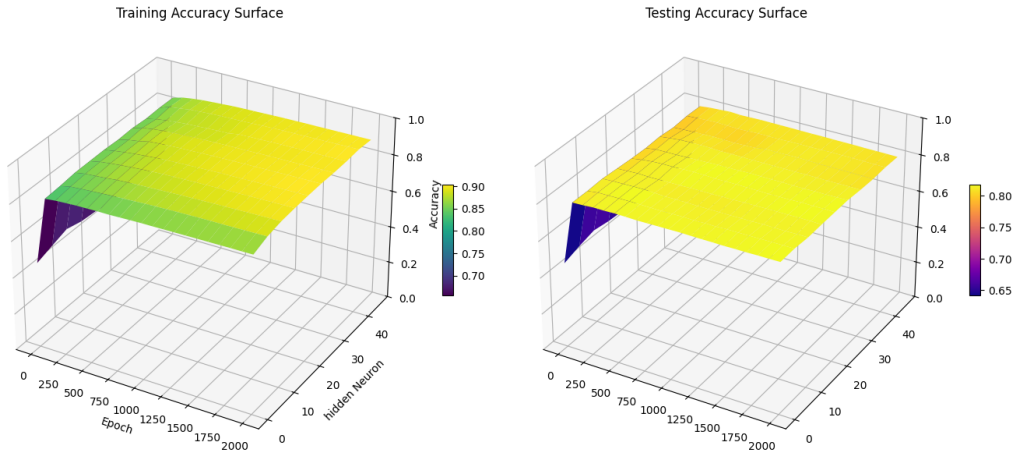


Figure 4.6: Training and testing accuracy surfaces for online (stochastic) learning.

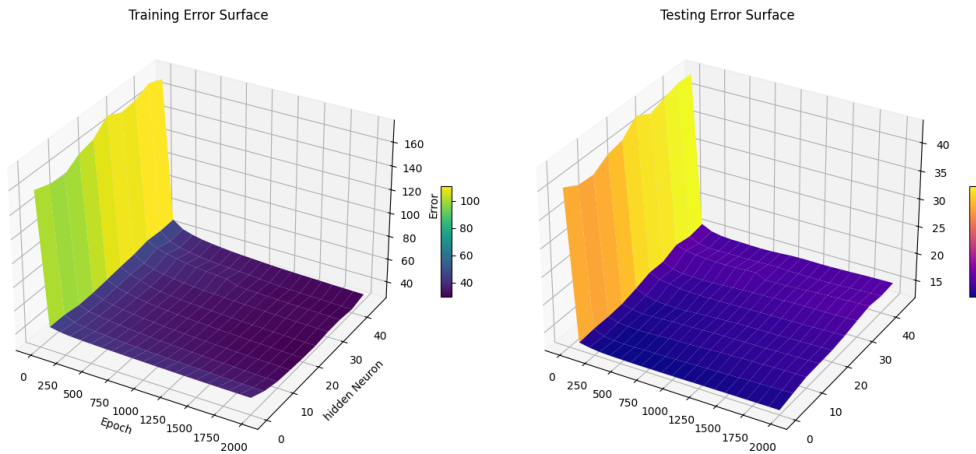


Figure 4.7: Training and testing error surfaces for online learning.

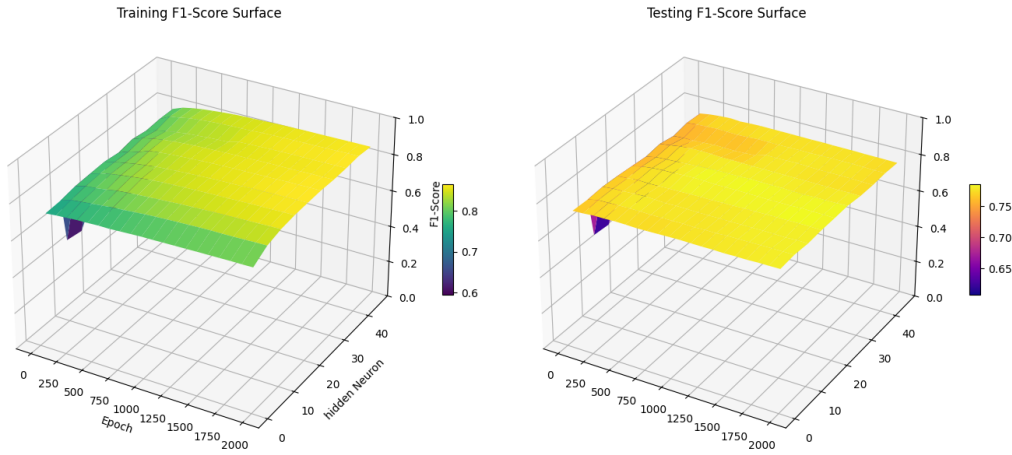


Figure 4.8: Training and testing F1-score surfaces for online learning.

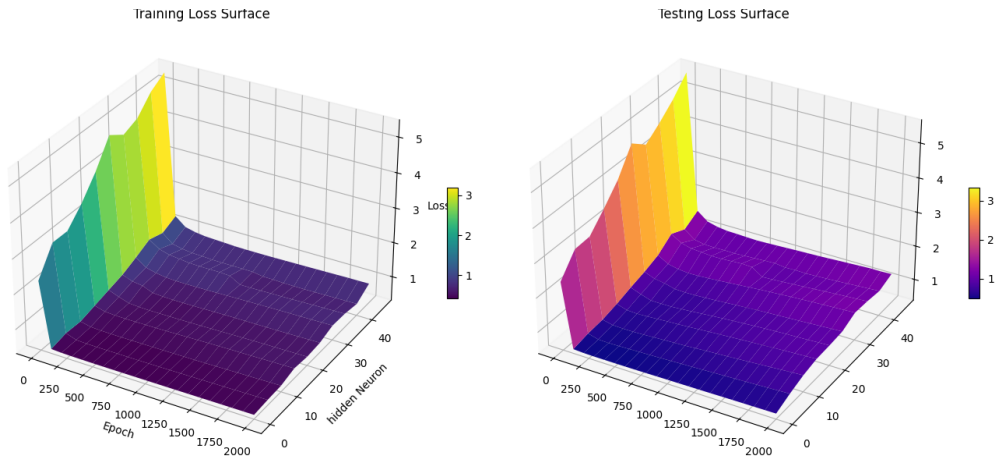


Figure 4.9: Training and testing cross-entropy loss surfaces for online learning.

Online learning exhibits faster initial convergence, reflecting the high gradient-update frequency. The test surfaces are smoother than their offline counterparts and show less evidence of over-fitting at large n_h , consistent with the regularising effect of gradient noise.

4.6 Training Dynamics

4.6.1 Offline Training Convergence

Figure 4.10 plots the training error and cross-entropy loss per epoch for the best-performing offline configuration. The loss decreases sharply during the first ~ 300 epochs and then flattens, indicating that the optimiser has reached a region near a local minimum. Residual oscillation in the error curve reflects the discrete nature of the classification error metric.

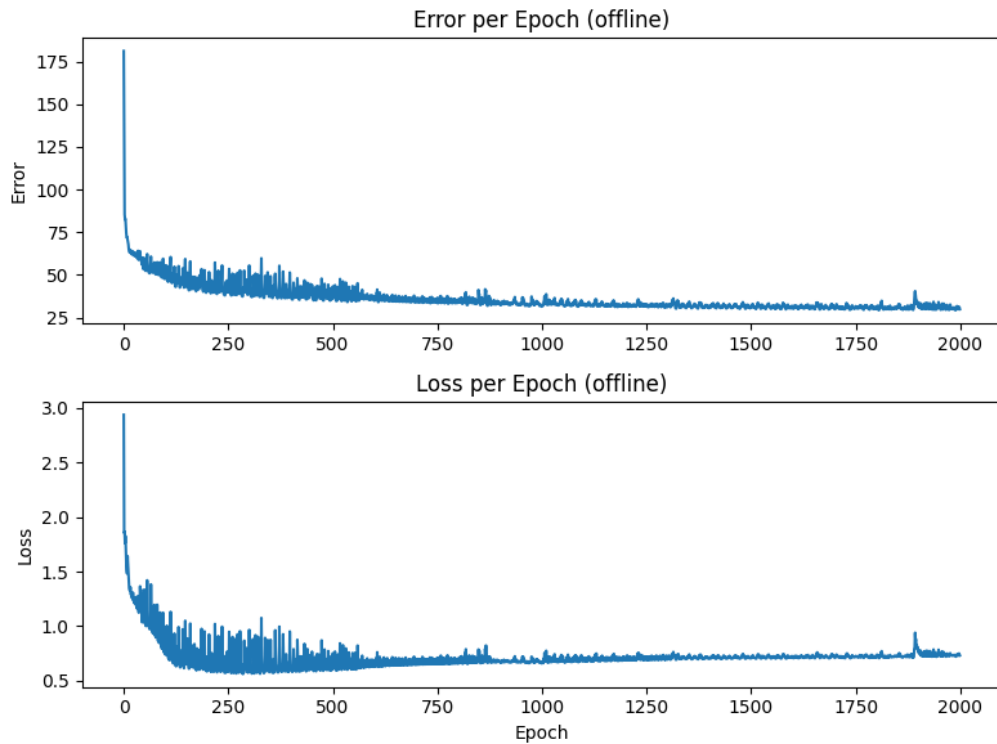


Figure 4.10: Epoch-by-epoch training error (top) and cross-entropy loss (bottom) for the best-performing offline configuration.

4.6.2 Online Training Convergence

Figure 4.11 shows the corresponding curves for online learning. The convergence profile is noticeably smoother, and the final training error is lower than for the offline variant, which reflects the higher effective number of weight updates per epoch in stochastic mode.

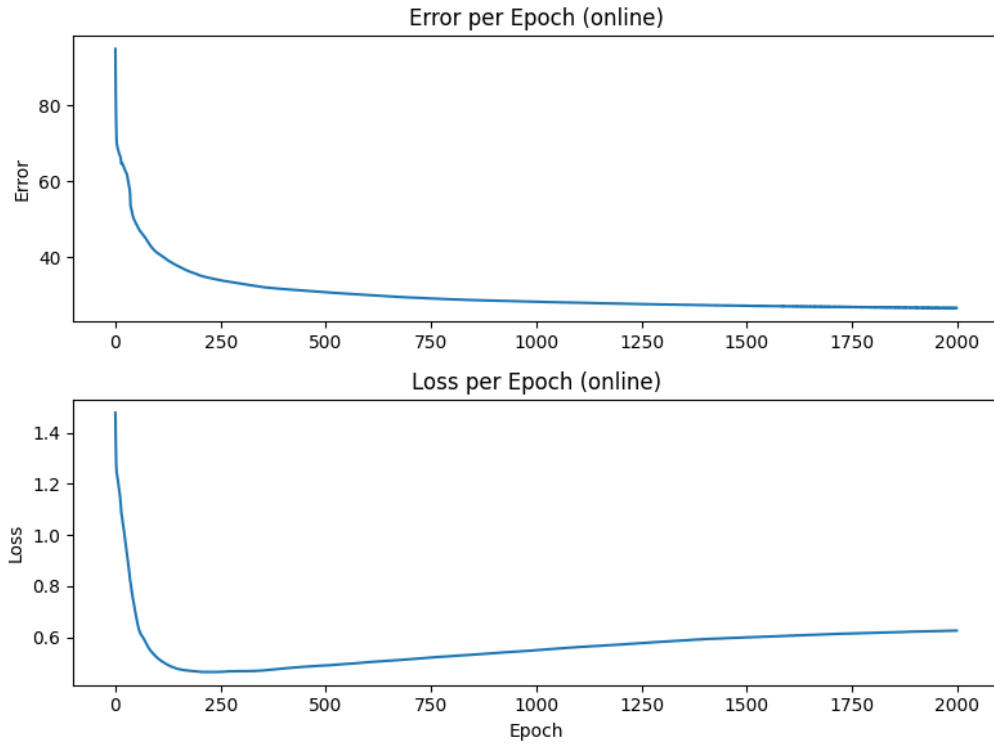


Figure 4.11: Epoch-by-epoch training error (top) and cross-entropy loss (bottom) for the best-performing online configuration.

4.7 ROC Analysis

The Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC) provide a threshold-independent assessment of classifier discriminative ability. Figure 4.12 compares the ROC curves for the best offline and online models.

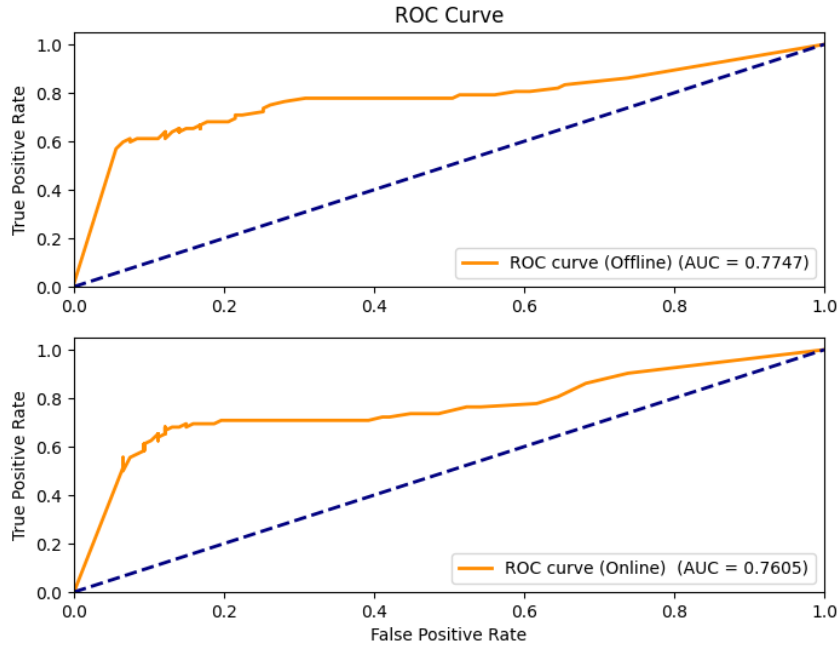


Figure 4.12: ROC curves for the offline (AUC = 0.7747) and online (AUC = 0.7605) models on the Titanic test set. The dashed diagonal represents a random classifier (AUC = 0.5).

Both models achieve AUC values well above the random baseline, confirming the network’s ability to discriminate survivors from non-survivors. The offline model achieves a marginally higher AUC (0.7747 vs. 0.7605), suggesting that the smoother gradient estimates provided by batch updates slightly improve the calibration of the output probabilities, even though online learning achieves a higher raw classification accuracy. This discrepancy is expected: accuracy is a threshold-sensitive metric, while AUC integrates performance across all thresholds.

4.8 Discussion

The neural network classifier demonstrates solid performance on the Titanic benchmark. Key observations are:

- Both learning strategies converge reliably within 2000 epochs for $n_h \geq 5$.
- Online learning converges faster and tends toward slightly higher accuracy, making it preferable when training time is a constraint.
- Offline learning yields better-calibrated probability outputs (higher AUC), which is advantageous in applications where the decision threshold must be tuned post-training.

- Hidden-layer size around $n_h = 20$ offers the best generalisation trade-off; larger networks improve training accuracy without proportional test improvement.

Chapter 5

Prototype Implementations

5.1 Overview

Two MATLAB prototype implementations were developed to provide visual and conceptual insight into the core mechanisms used in Tasks 1–2. These prototypes are deliberately simplified — they operate on low-dimensional synthetic data — so that the geometric behaviour of the algorithms can be directly observed and animated.

5.2 Competitive Learning Prototype (`Competitive.m`)

5.2.1 Setup and Algorithm

The competitive learning prototype initialises 81 neurons on a 2-D grid of data points arranged on the uniform lattice $\{-0.8, -0.6, \dots, 0.8\}^2$. Neuron weight vectors are initialised randomly in $[-0.5, 0.5]^2$.

At each iteration, a data point $\mathbf{x}^{(i)}$ is drawn uniformly at random and the K -nearest neurons are identified. The K -nearest neighbours and the BMU receive a weighted update:

$$\mathbf{w}_k \leftarrow \mathbf{w}_k + \frac{\alpha}{2}(\mathbf{x}^{(i)} - \mathbf{w}_k), \quad k \in \mathcal{N}_K(k^*), \quad (5.1)$$

with an additional factor-of-2 update applied specifically to the BMU k^* . The neighbourhood size K starts at $\lceil 1.5 \log_2 N \rceil$ (where N is the number of neurons) and is decremented periodically according to a geometrically increasing schedule, annealing to $K = 1$ (pure winner-take-all) by the end of training. The learning rate α decays multiplicatively at each step.

5.2.2 Animated Results

The simulation produces the animated GIF file `k-nearest-neighbors.gif` (Figure 5.1), which shows the migration of neuron weight vectors (circles) toward the data points (stars)

over 10 000 iterations.

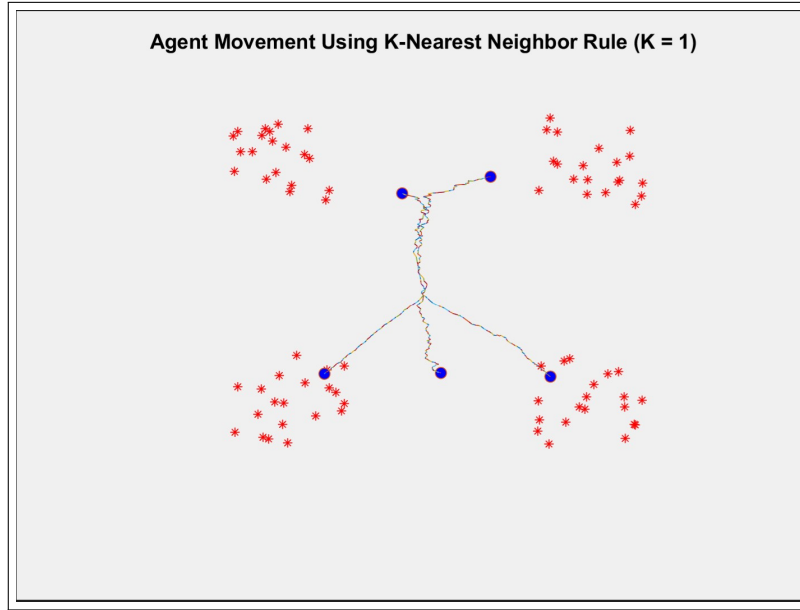


Figure 5.1: Still frame from the competitive learning animation (`k-nearest-neighbors.gif`). Circles represent neuron weight vectors migrating toward the data distribution (stars) under the K -nearest-neighbour adaptation rule.

A second animation (`k-nearest-neighbors_1.jpg`) visualises the effect with trajectory lines drawn between successive neuron positions, making the migration paths explicit.

5.2.3 Key Observations

- With large K , neurons collectively sweep toward data-dense regions, avoiding local dead-neuron traps.
- As K decreases toward 1, neurons settle into stable positions and fine-tune their representation of local data clusters.
- The optional *obsolete-neuron reallocation* mechanism (disabled in the default configuration) would reset neurons that have been selected fewer than 5% of the expected share, preventing dead neurons.

5.3 SOM Prototype (`Cluster.m`)

5.3.1 Setup and Algorithm

The SOM prototype operates on four Gaussian clusters of 20 points each, centred at $(\pm 2, \pm 2)$. Five neurons are initialised at the origin. The update rule mirrors the competitive learning prototype but applies to a clustering scenario where the spatial neighbourhood

on the map lattice is implicit in the K -nearest-neighbour selection. The neighbourhood parameter K starts at 2 and decreases to 1 after 500 iterations.

5.3.2 Animated Results

The simulation saves the trajectory of all five neurons as a GIF (Figure 5.2), with trajectory lines connecting consecutive positions to make the convergence path visible.

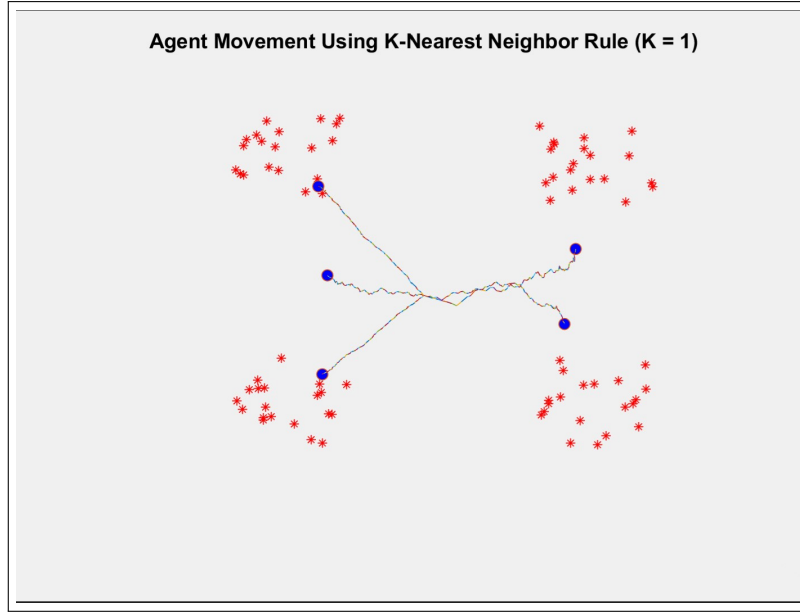


Figure 5.2: Still frame from the SOM prototype animation (`k-nearest-neighbors_1.jpg`). The five neurons (blue circles) migrate toward the four Gaussian clusters (red stars); trajectory lines trace the convergence paths.

5.3.3 Key Observations

- With five neurons for four clusters, the map reaches a configuration where approximately one neuron serves each cluster and the fifth neuron settles at an intermediate position, demonstrating quantisation in an over-provisioned scenario.
- The trajectory lines reveal that neurons initially move rapidly toward the data, then slow as the learning rate decays and K decreases.
- The transition from $K = 2$ to $K = 1$ is accompanied by a visible change in dynamics: neurons become more selectively attracted to their nearest cluster.

Chapter 6

Conclusion

This project has explored competitive learning, self-organizing maps, and neural network classification through both theoretical investigation and empirical evaluation.

SOM Clustering (Task 1): The comparison between the 4×4 and 20×20 SOM architectures clearly demonstrates that representational capacity is the primary determinant of clustering quality. The 4×4 map, with only sixteen neurons for ten digit classes, produces mixed-class neurons and coarse topological organisation. The 20×20 map, with 400 neurons, achieves sharp intra-class clusters, smooth inter-class transitions, and substantially lower quantisation error, at the cost of dead neurons and higher computation. The emergence of dead neurons in the larger map also highlights the importance of matching map size to the intrinsic data dimensionality or employing dead-neuron reallocation strategies.

SOM Classification (Task 2): Extending the trained SOMs to a classification role via majority-vote labelling demonstrates that the 20×20 architecture is significantly more effective. The finer spatial resolution allows most neurons to specialise in a single digit class, producing more accurate and consistent label maps. These findings motivate the use of larger SOM architectures — or post-training refinement methods such as LVQ — when classification accuracy is the primary objective.

Titanic Classification (Task 3): The single-hidden-layer MLP proves effective for binary survival prediction, achieving AUC values of approximately 0.77 (offline) and 0.76 (online) on the test set. Online learning converges faster and achieves slightly higher classification accuracy, while offline (batch) learning provides marginally better probability calibration as measured by AUC. A hidden-layer size of around 20 neurons offers the best generalisation across both training strategies. These results confirm that even a shallow neural network, properly trained and regularised, can extract meaningful predictive structure from a mixed-type tabular dataset.

Prototypes: The MATLAB competitive learning and SOM prototypes successfully illustrate the core algorithmic mechanisms — neighbourhood cooperation, learning-rate decay, and progressive neighbourhood shrinkage — in an intuitive 2-D visualisation. The

animated GIF outputs provide a compelling demonstration of how neuron weight vectors migrate and converge, offering pedagogical value beyond what static figures can convey.

Future Directions: Several extensions could further improve the results reported here. For the SOM tasks, growing or adaptive map variants could automatically tune neuron count to data complexity. For the Titanic task, regularisation techniques (dropout, weight decay), deeper architectures, or ensemble methods could push performance above the single-hidden-layer ceiling. Feature engineering — such as extracting family group information or combining `Pclass` with `Fare` — could also yield improvements. Finally, integrating the SOM representation with a downstream neural network (a hybrid unsupervised–supervised pipeline) is a natural direction for combining the topological insights of Task 2 with the discriminative power of Task 3.

In summary, the experiments reported in this project provide concrete empirical evidence for the well-known theoretical advantages of increased representational capacity in unsupervised learning, and demonstrate the practical competitiveness of shallow neural networks for structured tabular classification when combined with careful preprocessing and hyperparameter selection.

Bibliography

- [1] T. Kohonen, “The self-organizing map,” *Proceedings of the IEEE*, vol. 78, no. 9, pp. 1464–1480, 1990.
- [2] S. Haykin, *Neural Networks: A Comprehensive Foundation*, 2nd ed. Prentice Hall, 1999.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [4] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [5] J. Vesanto and E. Alhoniemi, “Clustering of the self-organizing map,” *IEEE Transactions on Neural Networks*, vol. 11, no. 3, pp. 586–600, 2000.
- [6] B. Erickson et al., “Machine learning for medical imaging,” *RadioGraphics*, vol. 37, no. 2, pp. 505–515, 2017.